



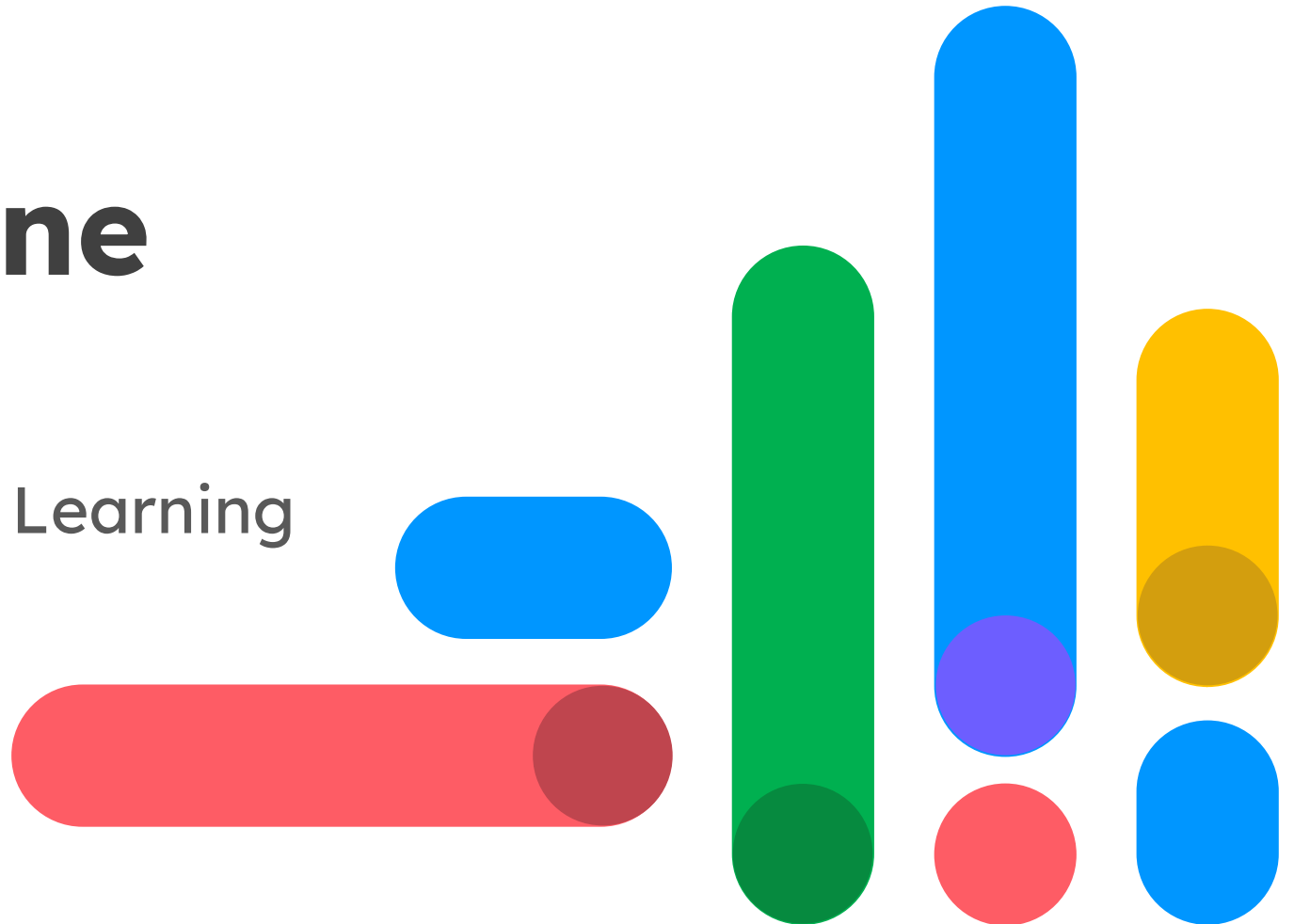
Conventional ML Methods

Applied Machine Learning

Classifier Models :: Ensemble Learning

Assistant Professor | **Chun-Hsiang Chan**

Department of Geography
National Taiwan Normal University



Outline

#GeoAI #Mobility #Geography #STDS

- 01 Logistic Regression
- 02 K-Nearest Neighbors
- 03 Naïve Bayes
- 04 Decision Tree
- 05 Bagging Methods
- 06 Random Forest
- 07 ExtraTree

- 08 Boosting Methods
- 09 AdaBoost
- 10 Gradient Boosting
- 11 XGBoost
- 12 CatBoost
- 13 LightGBM
- 14 Support Vector Machine

Bagging Methods

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

Bagging methods are from bootstrap aggregating, indicating *bagging* = *bootstrap* + *aggregating*.

Given a training dataset, $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$, so we have N observations.

We did randomly sampling N observations with replacement, then we got $D^{(1)}, D^{(2)}, \dots, D^{(M)}$.

For each sample ($D^{(m)}$), we may train a decision tree ($T_m(x)$); therefore, these trees may be trained independently and parallel.

Bagging Methods

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

For a classification method, prediction will be ...

$$\hat{y} = \text{mode}\{T_1(x), T_2(x), \dots, T_m(x)\}$$

Tree ID	1	2	3	4	5	6
Prediction	1	1	0	1	1	0

So, the answer $\hat{y} = 1$, since we have four 1s.

Solution: Train trees on different bootstrap samples and average them.

Remaining problem: Trees can still be highly correlated.

Bagging Methods

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

Due to sampling with replacement, all trees are not really independent. Furthermore, if one feature has very strong impact, the first split of most trees might be the same.

Given a correlation (ρ) between trees, we combine the variance of each tree is σ^2 .

So, the average variance is

$$Var(\bar{T}) = \rho\sigma^2 + \frac{1-\rho}{M}\sigma^2$$

When M approaches to ∞ , $Var(\bar{T})$ is $\rho\sigma^2$.

Bagging works better individual trees are less correlated.

Random Forest

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

Random Forest

A random forest is a meta estimator that fits a number of decision tree classifiers on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control overfitting.

random forest = bagging + random feature selection

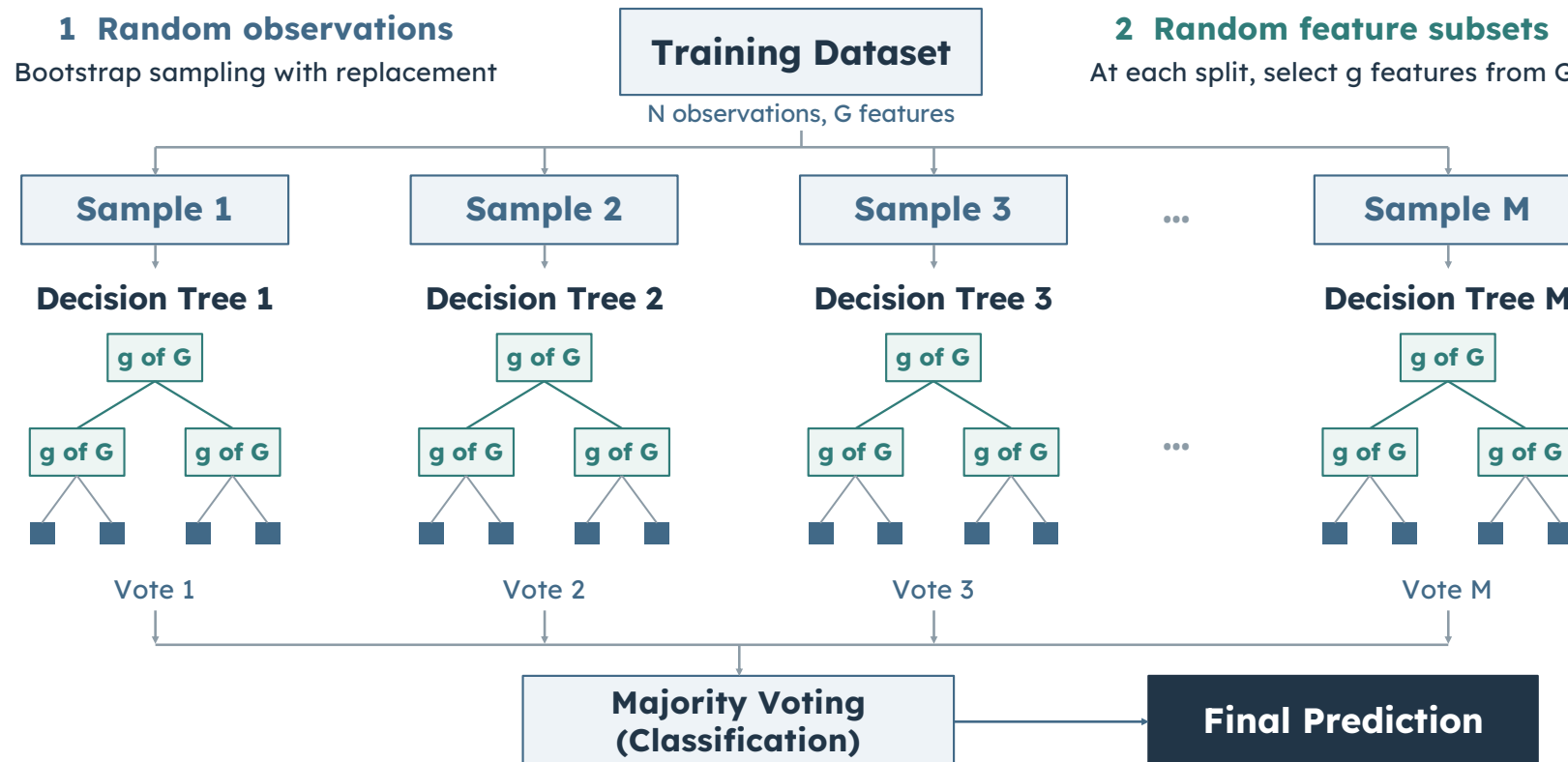
Given 100 features ($M = 100$), bagging considers all features and find the best feature for splitter.

But in random forest, we randomly select g features for each split (node), where $g < 100$ and find the best splitter.

Random Forest

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

Hence, we may use the following figure to explain it.



Random Forest

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

Out-of-Bag Samples (OOB)

When a Random Forest builds each decision tree, it uses **Bootstrap Sampling** (random sampling with replacement):

Suppose the training set contains N samples. The algorithm randomly draws 1 sample at a time, repeats this N times, and forms the training subset for a single tree.

Random Forest

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

Out-of-Bag Samples (OOB)

Mathematically, as N grows large, the probability of a specific sample **never being selected** is approximately:

$$\lim_{n \rightarrow \infty} \left(1 - \frac{1}{N}\right)^N = \frac{1}{e} \approx 36.8\%$$

This means that for every tree built, roughly **37% of the original training samples are left out**. These unselected samples are called **Out-of-Bag (OOB) samples**

Random Forest

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

Out-of-Bag Samples (OOB)

Advantages:

- **Data Efficiency:** No need to reserve a separate validation set (100% of data can be used for training).

Limitations:

- **Algorithm-Specific:** Only applies to Bagging-based models with bootstrap sampling (not for XGBoost/GBDT).
- **Sample-Size Sensitive:** Less reliable or stable on very small datasets.

Random Forest

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

Hyperparameters

class_weight: Weights associated with classes in the form of weight. If not given, all classes are supposed to have weight one.

The “**balanced**” mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data.

The “**balanced_subsample**” mode is the same as “**balanced**” except that weights are computed based on the bootstrap sample for every tree grown.

Random Forest

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

Hyperparameters

oob_score: it is a built-in validation metric used in Random Forest models.

Its greatest advantage is that **it estimates the model's generalization error on unseen data for free**, without requiring a separate validation set or performing K-fold Cross-Validation.

Extra-Trees

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

This class implements a meta estimator that fits a number of randomized decision trees (a.k.a. extra-trees) on various sub-samples of the dataset and uses averaging to improve predictive accuracy and control overfitting.

In contrast to random forests, Extra-Trees select random thresholds for each feature to enhance model generalizability on unseen test data instead of finding the best one from all candidate thresholds.

Extra Trees

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

But in `scikit-learn`, the hyperparameter of `bootstrap` is **False** as default.

ExtraTrees = Random Forest + Random Threshold

Since Extra-Trees conduct random threshold, the variance (correlation) might decline with an increase in randomness.

Extra Trees

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

Summary

	Bagging	Random Forest	Extra Trees
Multiple trees	V	V	V
Bootstrap samples	Usually	Usually	Optional; sklearn default
Random feature subset	Not inherent to basic bagging	V	V
Random thresholds	X	X	V
Main goal	Reduce variance	Reduce variance + tree correlation	Increase diversity/randomization
Trees trained	Parallel	Parallel	Parallel
Aggregation	Vote / Average	Vote / Average	Vote / Average
Randomness	Samples	Samples + Features	Features + Thresholds (+ optional samples)

Boosting Methods

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

As we mentioned before, a bagging method is

bagging = independent learners + mode (average)

All bagging methods want to do is reduce the variance (correlation) between trees; however, the 2nd tree does not learn anything or know anything from the 1st tree.

To avoid generating several useless trees, we introduce a new approach, called boosting method,

boosting = sequential learners + error correction

to have a better outcome from learning the errors of previous trees.

Boosting Methods

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

Boosting combines a sequence of weak learners, where each new learner focuses on correcting the errors made by the previous learners.

	Bagging	Boosting
Training	Parallel	Sequential
Learners	Independent	Dependent
Main purpose	Reduce variance	Improve weak learners / reduce errors
Each new model	Independent	Corrects previous errors
Combination	Average / Vote	Weighted/additive combination
Typical base learner	Deep/unstable trees possible	Shallow trees often used
Examples	Random Forest	AdaBoost, Gradient Boosting

AdaBoost

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

AdaBoost

Here, we introduce the first boosting method, which is also the simplest one, called AdaBoost.

AdaBoost achieves a higher prediction via increasing the weights of misclassified observations, since misclassified observations receive greater weights, forcing the next learner to pay more attention to them.

AdaBoost

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

Let's use an example to understand how AdaBoost works.

Initial Conditions

ID	x	y	Weight
A	1	-1	0.20
B	2	-1	0.20
C	3	+1	0.20
D	4	+1	0.20
E	5	-1	0.20

$$w_1^{(1)} = \frac{1}{N}; w_i = 0.2$$

Given a $h_1(x)$

ID	True	$h_1(x)$	Correct?
A	-1	-1	V
B	-1	-1	V
C	+1	+1	V
D	+1	+1	V
E	-1	+1	X

Only E is incorrect!

$$\epsilon_t = \sum_{i=1}^N w_i^{(t)} I(y_i \neq h_t(x_i)) \Rightarrow \epsilon_1 = 0.2$$

AdaBoost

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

Then, we need to evaluate how good is the weak learner?

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

Take $\epsilon_1 = 0.2$ in the above equation, then we got

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{0.8}{0.2} \right) = \frac{1}{2} \ln 4 = 0.693$$

It means that the lower ϵ_t we have, the higher α_t we get, implying the more confidence we trust this learner. Meanwhile, the more accurate weak learner, the more impact on the final ensemble.

AdaBoost

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

From a different point of view, if our weak learner just have a perfect prediction, which means no incorrect prediction.

$$\epsilon_t = \sum_{i=1}^N w_i^{(t)} I(y_i \neq h_t(x_i)) \Rightarrow \epsilon_t = 0$$

Then, the weak learner weight will be ...

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right) = \frac{1}{2} \ln \left(\frac{1 - 0}{0} \right) = +\infty$$

So, it turns out this learner is a perfect learner.

AdaBoost

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

If our weak learner just have a half/half prediction, that means $\epsilon_t = 0.5$.

Thus, the weak learner weight will be ...

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right) = \frac{1}{2} \ln \left(\frac{1 - 0.5}{0.5} \right) = \frac{1}{2} \ln \left(\frac{0.5}{0.5} \right) = 0$$

So, a zeros of weak learner weight means a random guessing.

$\epsilon_t = 0$	$0 < \epsilon_t < 0.5$	$\epsilon_t = 0.5$	$\epsilon_t > 0.5$
$\alpha_t = +\infty$	$\alpha_t > 0$	$\alpha_t = 0$	$\alpha_t < 0$
Perfect learner	Better than random	Random guessing	Worse than random

AdaBoost

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

Back to the modeling process, we have to update the sample weight, learning from the previous prediction result. Thus, we have ...

$$w_i^{(t+1)} = \frac{w_i^{(t)} \exp[-\alpha_i y_i h_i(x_i)]}{Z_t}$$

If this sample was predicted correctly, then

$$y_i h_i(x_i) = +1 \Rightarrow e^{-\alpha} \Rightarrow e^{-0.693} = 0.5 \Rightarrow 0.2 \times 0.5 = 0.1$$

If this sample was predicted incorrectly, then

$$y_i h_i(x_i) = -1 \Rightarrow e^{+\alpha} \Rightarrow e^{0.693} = 2 \Rightarrow 0.2 \times 2 = 0.4$$

AdaBoost

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

So, we obtain the sample weights of correct and incorrect predictions are 0.1 and 0.4, respectively.

ID	Weight	Correct?	New Weight	Normalized New Weight
A	0.20	V	0.1	0.125
B	0.20	V	0.1	0.125
C	0.20	V	0.1	0.125
D	0.20	V	0.1	0.125
E	0.20	X	0.4	0.500

Pay attention here, since E has a higher sample weight!

AdaBoost

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

After several iterations, we have finished the learning process of model. then, how to predict

For each learner, it has its owned α_t , thus

$$H(x) = \text{sign} \left[\sum_{t=1}^T \alpha_t h_t(x) \right]$$

For example, three learners have $\alpha_1 = 0.8, \alpha_2 = 0.5, \alpha_3 = 0.2$ and predict $h_1 = +1, h_2 = -1, h_3 = +1$

Hence, the weighted vote will be

$$0.8(+1) + 0.5(-1) + 0.2(+1) = 0.5 \Rightarrow H(x) = +1$$

AdaBoost

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

AdaBoost concentrated on “which observations were misclassified?” But, should we have to correct by changing observation weights?

Or, being much more general, **in which direction should the current model change to reduce the loss function?**

Otherwise, the outliers or incorrectly labeled observations might directly change the model training path, which may fail in the testing dataset.

Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

From AdaBoost to Gradient Boosting: How Should the Next Learner Correct the Model?

AdaBoost

Misclassified observations → Increase their weights

$$w_i^{(t+1)} \propto w_i^{(t)} e^{-\alpha_t y_i h_t(x_i)}$$

AdaBoost tells the next learner which observations deserve more attention. But is reweighting observations the only way to correct previous errors?

Instead of asking “Which observations should receive more attention?”, can we ask “How should the current model change to reduce the loss?”

Here it comes ~~ Gradient Boosting!

Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

If we want to predict whether this person has a disease.

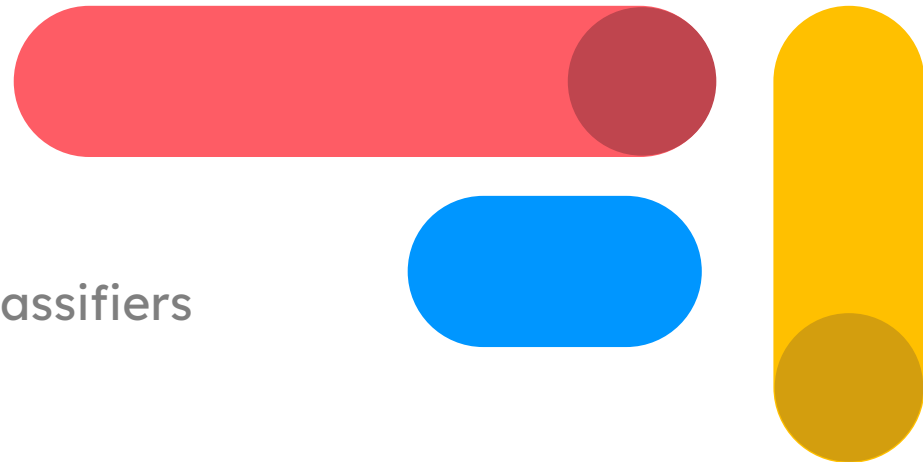
Thus, the output is a binary y .

$$y_i = \{0, 1\}$$

where $N_0 = 3, N_1 = 2$.

In Gradient Boosting Classifier, we usually adopt log-loss:

$$L = -[y \log(p) + (1 - p) \log(1 - p)]$$



ID	X	Disease (y)
A	1	0
B	2	0
C	3	0
D	4	1
E	5	1

Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

In binary classification, the additive scores in the model are usually log-odds scale. First, we first calculate the positive class probability:

$$p_0 = \frac{2}{5} = 0.4$$

Then, we transform it to log-odds:

$$F(x) = \log \frac{p(x)}{1 - p(x)} \Rightarrow F_0 = \log \left(\frac{p_0}{1 - p_0} \right) = \log \left(\frac{0.4}{0.6} \right) = -0.405$$

Via Sigmoid Transform,

$$p(x) = \frac{1}{1 + e^{-F(x)}} \Rightarrow p_i = \frac{1}{1 + e^{0.405}} = 0.4$$

Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

For each prediction of the first iteration,

ID	y_i	Initial (F_0)	p_i
A	0	-0.405	0.40
B	0	-0.405	0.40
C	0	-0.405	0.40
D	1	-0.405	0.40
E	1	-0.405	0.40

Without using any features, the model only knows that 40% of training data are positive cases.

Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

Then, we have to calculate the negative gradient

The log-loss was defined as

$$L_i = -[y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Compute the negative gradient the background's model score F .

$$r_i = -\frac{\partial L_i}{\partial F_i} = -\left(\frac{\partial L_i}{\partial p_i} \frac{\partial p_i}{\partial F_i}\right)$$

$$\frac{\partial L_i}{\partial p_i} = -\frac{y_i}{p_i} + \frac{1-y_i}{1-p_i} = -\frac{y_i(1-p_i)+p_i(1-y_i)}{p_i(1-p_i)} = \frac{-y_i+y_i p_i+p_i-p_i y_i}{p_i(1-p_i)} = \frac{p_i-y_i}{p_i(1-p_i)}$$

$$\frac{\partial p_i}{\partial F_i} = p_i(1 - p_i)$$

$$\therefore r_i = -\frac{\partial L_i}{\partial F_i} = -\left(\frac{\partial L_i}{\partial p_i} \frac{\partial p_i}{\partial F_i}\right) = \frac{p_i-y_i}{p_i(1-p_i)} p_i(1 - p_i) = y_i - p_i$$

Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

For the negative gradient, we get $r_i = y_i - p_i$

Since $y = 0, p = 0.4$, for A, B, & C we get $r = 0 - 0.4 = -0.4$

Since $y = 1, p = 0.4$, for D & E we get $r = 1 - 0.4 = +0.6$

ID	y_i	p_i	Negative Gradient ($y_i - p_i$)
A	0	0.40	-0.40
B	0	0.40	-0.40
C	0	0.40	-0.40
D	1	0.40	+0.60
E	1	0.40	+0.60

Why does “ $y - p$ ” mean here?

If $y = 1, p = 0.4$, then $y - p = +0.6$

Thus, the prediction needs to move **upward**.

Conversely, if $y = 0, p = 0.4$, then $y - p = -0.4$

That means the prediction needs to move **downward**.

Negative Gradient tells in which direction should the current prediction move to reduce the loss.

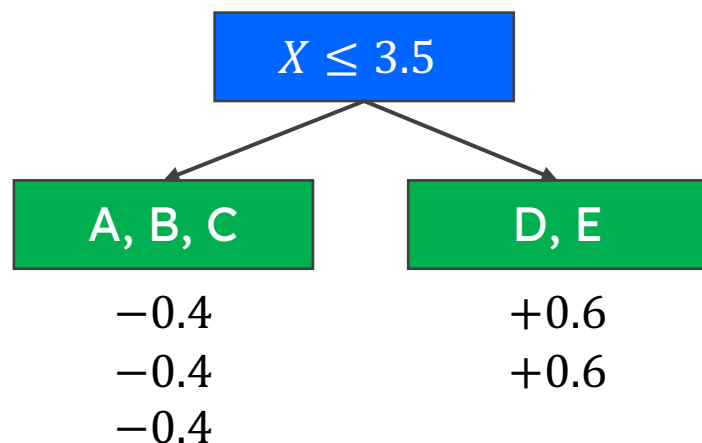
Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

What is the negative gradient in model?

From X , we have learn the pattern by $(y - p)$

We display the negative gradient in decision tree.



ID	X	Disease (y)
A	1	0
B	2	0
C	3	0
D	4	1
E	5	1

We could find that we may use $X \leq 3.5$ to divide into two parts.

Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

So, how does negative gradient changes the learning path of gradient boosting classifier?

Negative gradient, just like a slope, tells us which direction we will go to. Therefore, for Bernoulli log-loss, we have to compute the optimal update after establishing the tree, which tells us how many steps we need to move forward.

Usually, we may use Newton-style leaf update to represent

$$\gamma_j = \frac{\sum_{i \in R_j} (y_i - p_i)}{\sum_{i \in R_j} p_i (1 - p_i)}$$

Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

So, how determine the value of optimal update?

For j -th leaf of R_j , we have to find a correction γ_j that the loss of all observations in this leaf will be minimized.

$$\gamma_j = \operatorname{argmin}_{\gamma} \sum_{x_i \in R_j} L(y_i, F_{m-1}(x_i) + \gamma)$$

γ is the optimal step size for this leaf.

Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

All we have to do is to solve the equation.

$$\frac{\partial}{\partial \gamma} \sum_{i \in R_j} L(y_i, F_i + \gamma) = 0$$

$$\text{since } \frac{\partial L_i}{\partial F_i} = p_i - y_i$$

$$\frac{\partial L_i}{\partial F_i} = \sum_{i \in R_j} (p_i(\gamma) - y_i)$$

$$\therefore p_i(\gamma) = \sigma(F_i + \gamma)$$

$$\therefore \sum_{i \in R_j} [\sigma(F_i + \gamma) - y_i] = 0$$

However, the following equation is too difficult to solve.

$$\sigma(F_i + \gamma) = \frac{1}{1 + e^{-(F_i + \gamma)}}$$

Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

Therefore, we introduce **Newton-Raphson/ Second-order approximation**. The formula of Newton optimization is

$$\gamma = \frac{L'(\gamma)}{L''(\gamma)}$$

That is,

$$step = - \frac{\text{first derivative}}{\text{second derivative}}$$

As a result, where comes two questions: what is the first derivative and the second derivative?

Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

For the **first derivative of Bernoulli log-loss**,

$$\frac{\partial L_i}{\partial F_i} = p_i - y_i$$

For a leaf,

$$L' = \sum_{i \in R_j} (p_i - y_i)$$

Therefore,

$$-L' = \sum_{i \in R_j} (y_i - p_i) \quad \text{This is negative gradient!}$$

Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

For the **second derivative of Bernoulli log-loss**,

$$\frac{\partial^2 L_i}{\partial F_i^2} = \frac{\partial(p_i - y_i)}{\partial F_i} = \frac{\partial p_i}{\partial F_i}, \text{ since } y_i \text{ is constant}$$

$$\text{from Sigmoid derivative, } \frac{\partial p_i}{\partial F_i} = p_i(1 - p_i)$$

$$\frac{\partial^2 L_i}{\partial F_i^2} = p_i(1 - p_i)$$

For a leaf,

$$L'' = \sum_{i \in R_j} p_i(1 - p_i)$$

Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

Back to Newton formula, we can obtain ...

$$\gamma = \frac{L'}{L''} \Rightarrow \gamma_j = -\frac{\sum_{i \in R_j} (p_i - y_i)}{\sum_{i \in R_j} p_i (1 - p_i)} = \frac{\sum_{i \in R_j} (y_i - p_i)}{\sum_{i \in R_j} p_i (1 - p_i)}$$

Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

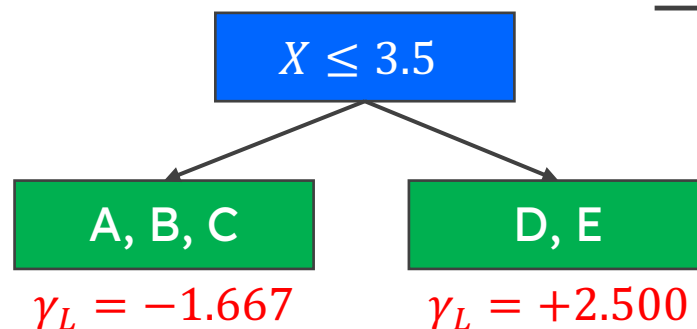
Now, we can get the γ of each leaf.

For the left leaf {A,B,C},

$$\gamma_L = \frac{\sum_{i \in R_L} (y_i - p_i)}{\sum_{i \in R_L} p_i(1 - p_i)} = \frac{-0.4 - 0.4 - 0.4}{3 \times 0.4(0.6)} = \frac{-1.2}{0.72} = -1.667$$

For the right leaf {D,E},

$$\gamma_R = \frac{\sum_{i \in R_R} (y_i - p_i)}{\sum_{i \in R_R} p_i(1 - p_i)} = \frac{0.6 + 0.6}{2 \times 0.4(0.6)} = \frac{1.2}{0.48} = 2.5$$



Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

In reality, we don't want to moving forward too much, because we are not sure the step we move is direct to the optimal solution, even though we have Bernoulli log-loss. To avoid missing the optimal solution, we have to add a learning rate, controlling the moving distance of each step. Given a learning rate (η) as 0.1,

$$F_1(x) = F_0(x) + \eta \gamma_j$$

For {A, B, C},

$$\begin{aligned} F_1 &= -0.405 + 0.1(-1.667) \\ &= -0.572 \end{aligned}$$

For {D, E},

$$\begin{aligned} F_1 &= -0.405 + 0.1(2.5) \\ &= -0.155 \end{aligned}$$

Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

Finally, we may calculate the probability

$$p = \frac{1}{1 + e^{-F}}$$

For {A, B, C},

$$p = \frac{1}{1 + e^{0.572}} \approx 0.361$$

For {D, E},

$$p = \frac{1}{1 + e^{0.155}} \approx 0.461$$

The probability **declines** since the ground truth is $y = 0$, therefore, the p needs to be **lower**.

The probability **increases** since the ground truth is $y = 1$, therefore, the p needs to be **larger**.

ID	y	Initial p	New p
A	0	0.400	0.361
B	0	0.400	0.361
C	0	0.400	0.361
D	1	0.400	0.461
E	1	0.400	0.461

Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

In the second run,

$$r_i^{(2)} = y_i - p_i^{(1)}$$

For {A, B, C}, we get $0 - 0.361 = -0.361$

For {D, E}, we get $1 - 0.461 = +0.539$

Then, $X \rightarrow r_i^{(2)}$

We have to train the second tree ($h_2(x)$) and new correction will be γ_{2j} ; then, we can use $F_2(x) = F_1(x) + \eta\gamma_{2j}$ and obtain new probability with sigmoid transform $F_2 \rightarrow p_2$. And keep the same procedure, ...

Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers

To sum up, we may integrate the whole process as follows:

$$F_0 = \log \frac{p}{1-p}$$

$$p_i = \sigma(F_i)$$

$$r_i = y_i - p_i$$

Train a tree to fit the negative gradients

$$F_m = F_{m-1} + \eta h_m$$

$$p_m = \sigma(F_m)$$

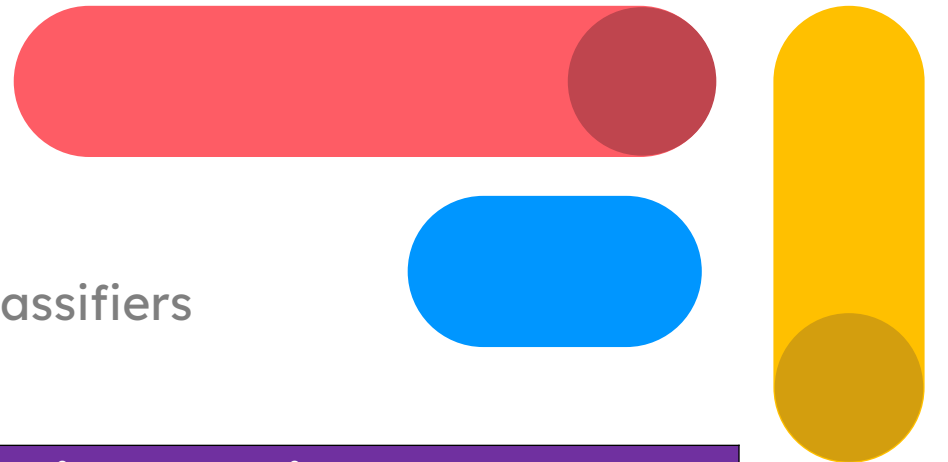
Calculate new negative gradients

$$r_i^{(m+1)} = y_i - p_i^{(m)}$$

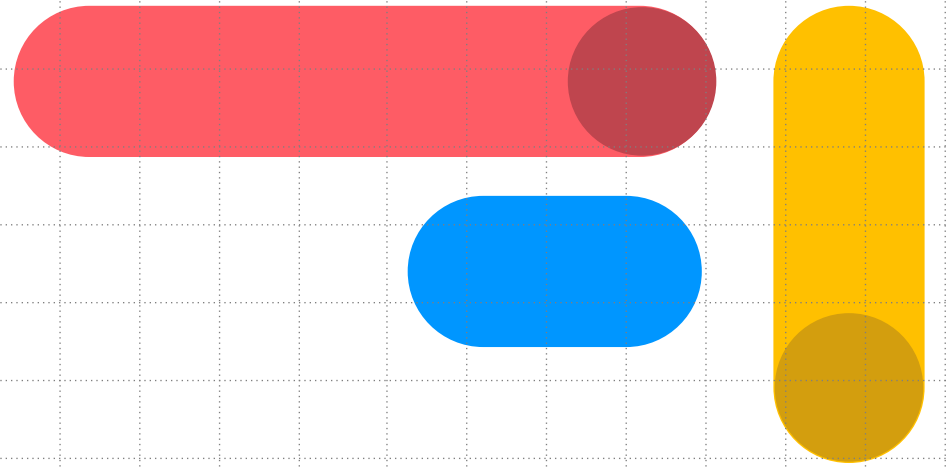
Repeat ... >>>

Gradient Boosting

Fundamental Classifiers | **Ensemble Learning** | Modern Classifiers



	AdaBoost	Gradient Boosting
Core idea	Focus on misclassified observations	Reduce a loss function
How?	Reweight observations	Fit negative gradients
Next learner learns	Hard/misclassified cases	Current model's correction direction
Model contribution	α_t	$\eta h_t(x)$
Loss perspective	Closely related to exponential loss	General differentiable loss
Base learners	Often decision stumps	Usually shallow trees
Training	Sequential	Sequential
Final model	Weighted vote	Additive model



The End

Thank you for your attention!

Email: chchan@ntnu.edu.tw

Website: <https://toodou.github.io/>